

USER MANUAL

BY

SFC Qorvex

UrbanTransit IQ

Transport Intelligence Dashboard — a year of a city bus network, read through seventeen screens

TechWiz 7 · Theme: TransitVerse Intelligence · Category: Data Science Intelligence Arena

Live demo: <https://urbantransit-iq.vercel.app>

Repository: <https://github.com/darike/UrbanTransitIQ>

Before you start

UrbanTransit IQ is a web application. Open it in any current browser — Chrome, Edge or Firefox. There are two ways to reach it:

- **The live URL, <https://urbantransit-iq.vercel.app>** — the hosted React dashboard. It signs you in through the built-in demo accounts and is the fastest way to look around.
- **The full local build** — start the FastAPI backend (uvicorn on port 8000) and the Vite dev server as described in the Installation Instructions, then open <http://localhost:5173>. With the backend running you also get real JWT sign-in, account creation with e-mail verification, password reset and the account roster.

The dashboard reads a 2.06-million-record synthetic transport dataset that has been cleaned, integrated and modelled twice — once with Spark/Spark MLlib and once with the Python data-science stack — so every screen shows analytical conclusions, not just charts. The live network map and the weather chip need an internet connection even when running locally, because map tiles and the Karachi weather feed come from public services.

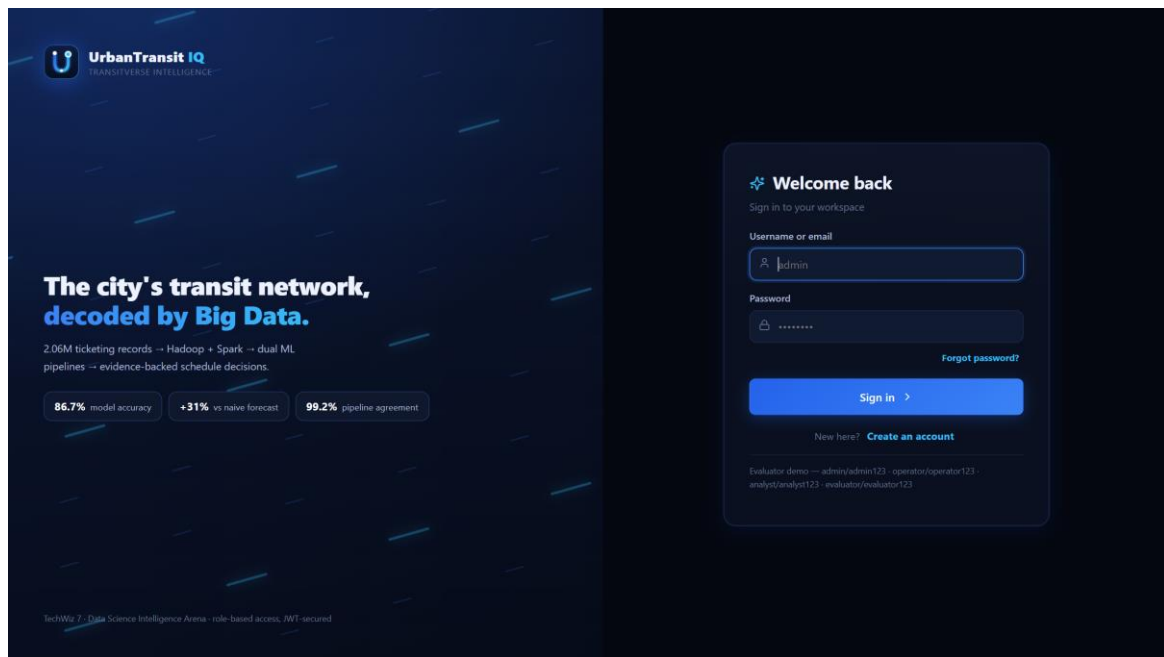
Signing in

Four accounts are provided, one per role. Each role reaches a different amount of the system, and the sign-in page lists them under the form.

Username	Password	What this role is for
operator	operator123	Transit Operator — the running of the network day to day: what is crowded, what is late, and what to do about it. No data-science internals and no exports.
analyst	analyst123	Data Analyst — everything the operator sees, plus the dual-pipeline model lab, data quality, Power BI and the CSV exports.
evaluator	evaluator123	Evaluator — a read-everything jury account: all analytics plus the HDFS & Spark Jobs screen. This is the account to use for marking.
admin	admin123	Administrator — everything, including infrastructure and the user roster on the User Management screen.

Use evaluator / evaluator123 if you are marking this project. It reaches every screen except User Management, which only the administrator sees.

With the backend running, sign-in is a real JWT exchange: the token is held for the session and every API call carries it, and the same role matrix is enforced again on the server, so a blocked role receives HTTP 403 even from a hand-crafted request. The sign-in page also offers account creation with an e-mailed verification code and a forgotten-password flow. If the backend is not reachable, the page says so and falls back to the four demo accounts so the dashboard remains usable.



The sign-in page. The four demo accounts are listed under the form.

How to move around

The sidebar

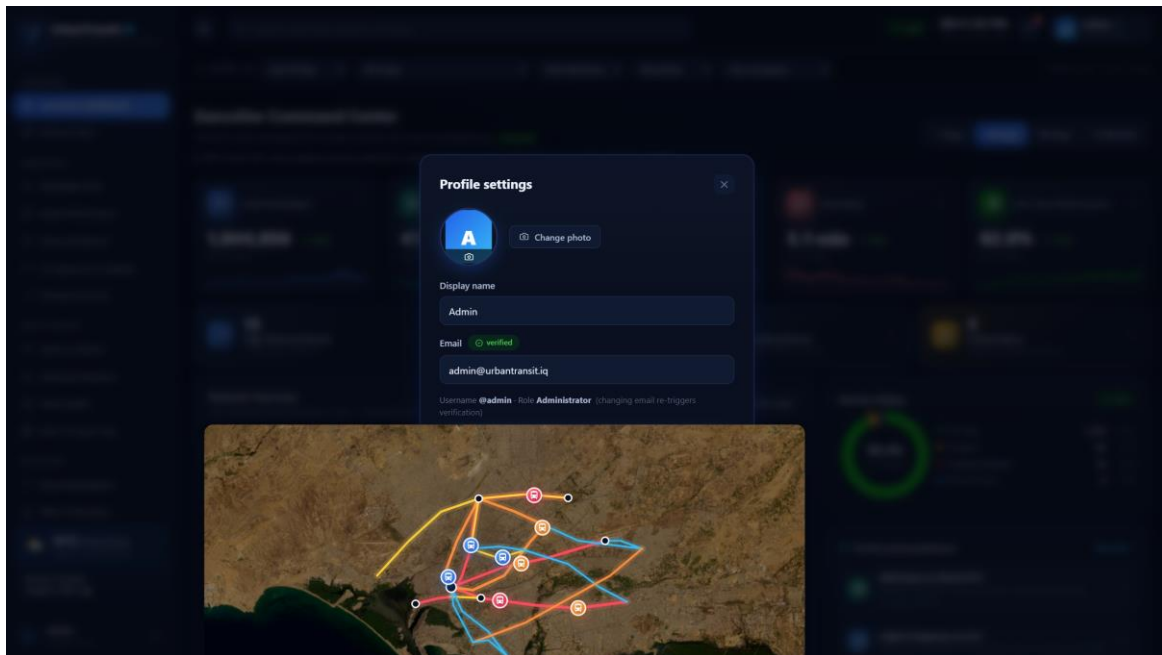
Seventeen entries down the left, in the order the work flows: Home, Executive Dashboard, Live Network Map, Passenger Flow, Route Performance, Delay Intelligence, Occupancy & Crowding, Demand Forecast, Spark vs Python, Anomaly Detection, Data Quality, HDFS & Spark Jobs, User Management, Recommendations, What-If Simulator, Reports & Export and Power BI. You only see the entries your role can open — the same matrix that guards the routes and the API also builds the menu.

The top bar

The top bar carries the page search, the live Karachi weather chip (from the Open-Meteo feed), and your profile. Click the avatar to open the profile card, change display details or sign out.



The top bar with the live weather chip.



The profile card.

Filters, sorting and legends

Screens that analyse routes and occupancy carry filter controls — route, occupancy band, time window — and a chart with a legend lets you click an entry to hide or show that series. Any table column heading with an arrow can be clicked to sort by it; click again to reverse.

The screens

Home

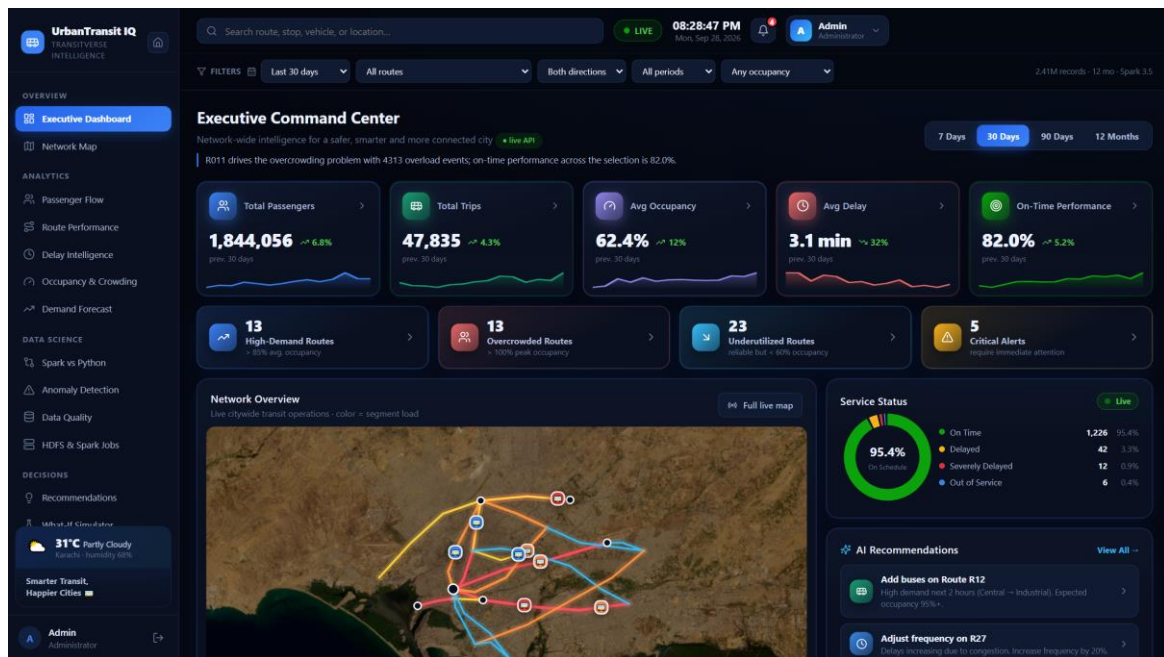
Who sees it: everyone, before and after sign-in.

The public landing page: what the project is, the pipeline from raw tickets to decisions, the dual-pipeline idea, and the roles. From here, Enter the dashboard takes you to sign-in — or straight to the Executive Dashboard if you are already signed in.

Executive Dashboard

Who sees it: all four roles. This is the screen the dashboard opens on.

The answer to “how is the network doing?” in one screen. Along the top are the headline figures — total passengers, total trips, average occupancy, average delay and on-time performance. Below them, passenger and service trends by hour split weekday against weekend, and ranked tables pick out the routes that deserve attention first.

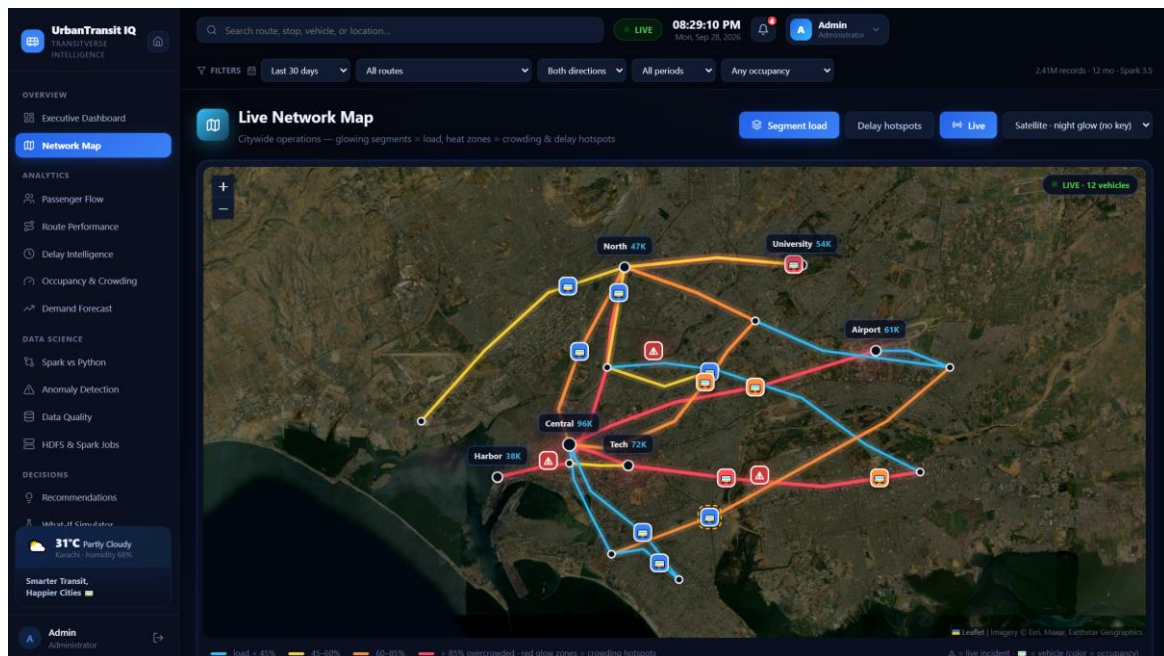


The Executive Dashboard.

Live Network Map

Who sees it: all four roles.

The network drawn on a real map (Leaflet). Stops are plotted by coordinate and sized by demand, routes are selectable, and hotspots stand out at a glance. Use it to turn a route number from a table into a place in the city.



The Live Network Map.

Passenger Flow

Who sees it: all four roles.

Where people travel and when. The busiest stops are ranked by boardings against alightings, the origin-destination matrix shows movements between the highest-volume stops (a Spark SQL aggregation), and

the network boarding profile by hour splits demand by direction — inbound dominates the morning peak, outbound the evening.

Route Performance

Who sees it: all four roles.

Every route ranked and scored. Utilisation, reliability and demand are read together so a very full route does not automatically score well, and the table can be re-sorted by any column to answer a different question.

Delay Intelligence

Who sees it: all four roles.

Delay read four ways: a day-by-hour heatmap that finds the troubled windows, the punctuality distribution across the five severity classes from On time to Severe, average and 90th-percentile delay by route, and the delay-severity model behind the predictions. The heatmap respects the route and occupancy filters.

Occupancy & Crowding

Who sees it: all four roles.

How full the vehicles are. Trips are banded from Low to Critical, the twelve-month trend shows whether crowding is growing, and the crowding-risk model scores the trips most likely to run overcrowded in the next 24 hours — published as a probability, so the borderline cases are visible rather than hidden behind a verdict.

Demand Forecast

Who sees it: all four roles.

Passenger demand forecast against what actually happened. The actual-versus-forecast panel is the one to read first: it shows where the model tracked and where it did not, which is more honest than a single accuracy number.



The Demand Forecast screen.

Spark vs Python

Who sees it: analyst, evaluator and admin.

The dual-pipeline comparison the specification asks for. The same problems are modelled twice — once in Spark MLlib, once in the Python data-science stack, trained separately — and this screen measures whether the two agree: agreement by predicted class, correlation, and the disagreeing cases, which concentrate at class boundaries. Two independent builds landing on the same answer is stronger evidence than either build alone.



The dual-pipeline comparison.

Anomaly Detection

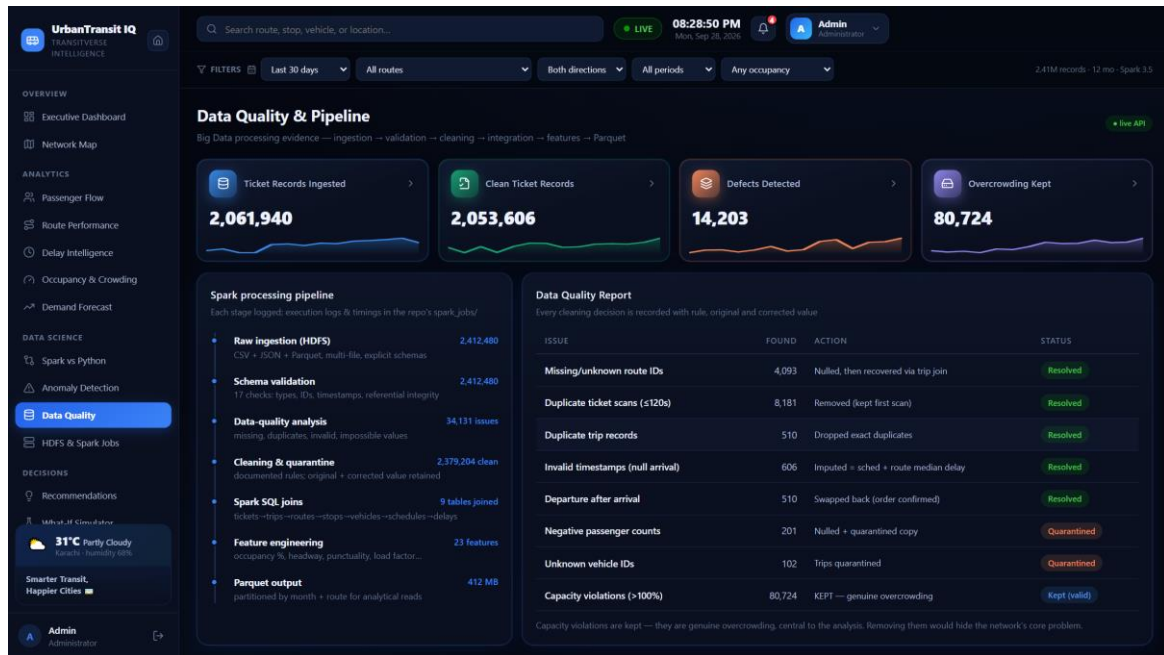
Who sees it: all four roles.

Everything the pipeline flagged as unusual — demand spikes and collapses, extreme delays, abnormal occupancy — listed with the behaviour that triggered the flag, so an anomaly is an explained event rather than a bare row.

Data Quality

Who sees it: analyst, evaluator and admin.

The quality battery: how many ticket records were ingested, how many survived cleaning, every defect class the pipeline detected, and what cleaning did about each — corrected, deduplicated or quarantined. Checks that found nothing are listed too, because a check that never fires is still evidence.



The Data Quality screen.

HDFS & Spark Jobs

Who sees it: evaluator and admin.

The Big Data infrastructure behind the dashboard: HDFS usage, the file tree, the Spark jobs that ran in the last day and the Parquet datasets they produced, with the terminal evidence of the runs. This is the screen that shows the pipeline is real.

```
hamza@DESKTOP - MSL2 Ubuntu - Hadoop 3.4.1 - single-node HDFS

$ hdfs dfs -ls -h /urbantransit/raw_data && hdfs dfsadmin -report
=== UrbanTransit IQ - HDFS evidence (Mon Sep 28 20:07:58 PDT 2026) ===

$ jps
17252 DataNode
33175 Jps
17195 NameNode

$ hdfs dfs -ls -h /urbantransit/raw_data
Found 12 items
-rw-r--r-- 1 hamza supergroup 38.1 M 2026-09-28 20:07 /urbantransit/raw_data/delays.json
-rw-r--r-- 1 hamza supergroup 2.6 M 2026-09-28 20:07 /urbantransit/raw_data/gps_events.csv
-rw-r--r-- 1 hamza supergroup 17.4 M 2026-09-28 20:07 /urbantransit/raw_data/passenger_counts.csv
-rw-r--r-- 1 hamza supergroup 2.4 M 2026-09-28 20:07 /urbantransit/raw_data/passengers.csv
-rw-r--r-- 1 hamza supergroup 28.7 K 2026-09-28 20:06 /urbantransit/raw_data/route_stops.csv
-rw-r--r-- 1 hamza supergroup 8.8 K 2026-09-28 20:06 /urbantransit/raw_data/routes.csv
-rw-r--r-- 1 hamza supergroup 2.1 K 2026-09-28 20:07 /urbantransit/raw_data/schedules.csv
-rw-r--r-- 1 hamza supergroup 9.6 K 2026-09-28 20:07 /urbantransit/raw_data/service_calendar.csv
-rw-r--r-- 1 hamza supergroup 22.9 K 2026-09-28 20:06 /urbantransit/raw_data/stops.csv
-rw-r--r-- 1 hamza supergroup 128.7 M 2026-09-28 20:07 /urbantransit/raw_data/tickets.csv
-rw-r--r-- 1 hamza supergroup 67.5 M 2026-09-28 20:07 /urbantransit/raw_data/trips.csv
-rw-r--r-- 1 hamza supergroup 11.0 K 2026-09-28 20:06 /urbantransit/raw_data/vehicles.csv

$ hdfs dfs -du -h -s /urbantransit
344.6 M 344.6 M /urbantransit

$ hdfs dfsadmin -report
```

HDFS & Spark Jobs, with terminal evidence.

User Management

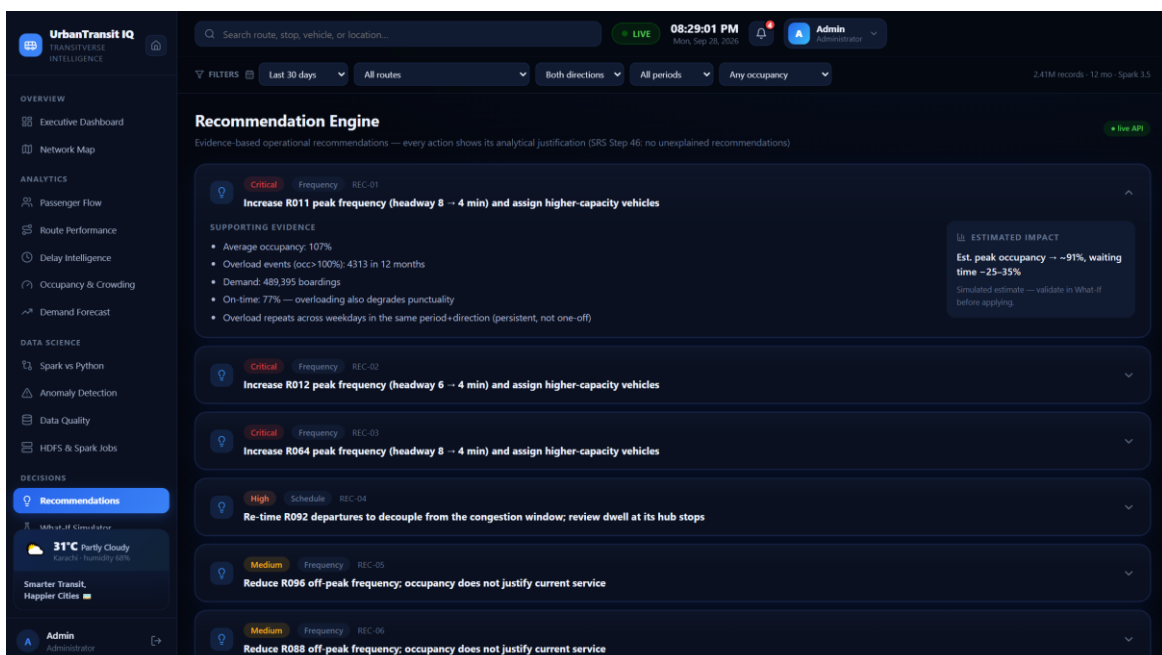
Who sees it: administrator only.

The account roster: every user, their role and status, with creation and deletion. The same role matrix that builds the sidebar is what this screen edits, so the permission model and its administration cannot disagree.

Recommendations

Who sees it: all four roles.

Where the system tells you what to do. Each recommendation — add capacity here, adjust frequency there — carries the evidence behind it rather than a bare verdict, so it can be checked against the screens it came from.

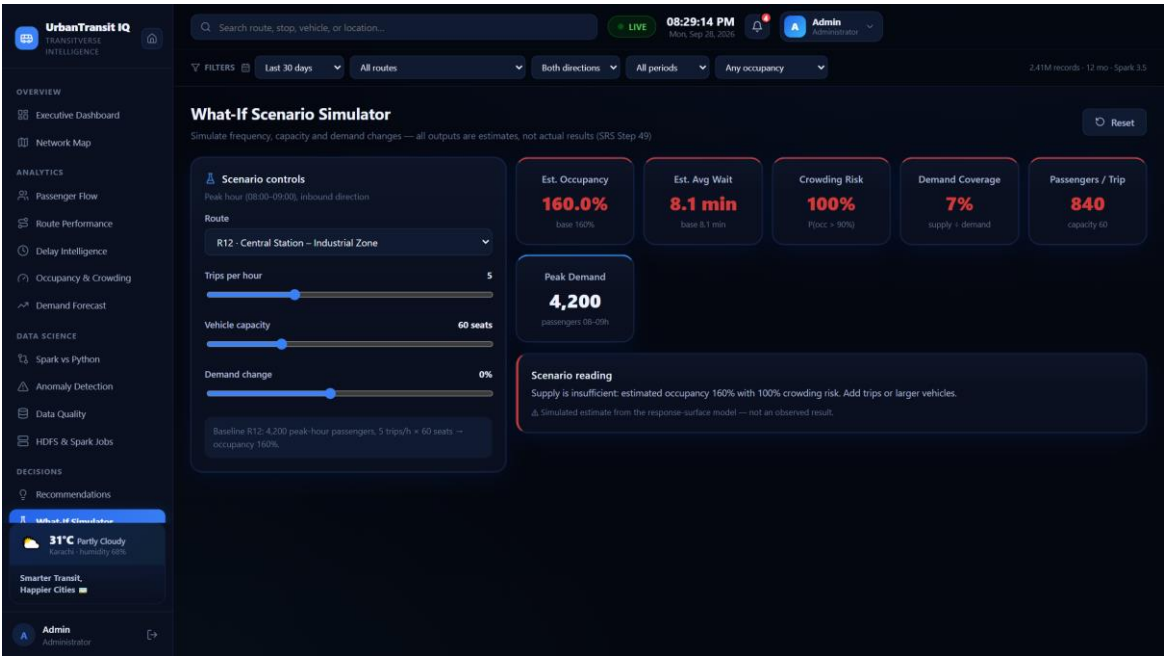


The Recommendation Engine.

What-If Simulator

Who sees it: all four roles.

Ask what happens if you act on a recommendation. Pick a route, then move trips per hour, vehicle capacity or expected demand, and the panel estimates occupancy, average wait and crowding risk. Every figure is an estimate with demand held fixed — an upper bound on the relief, not a promise.



The What-If Scenario Simulator.

Reports & Export

Who sees it: analyst, evaluator and admin.

Every analytical dataset the project produces, listed by subject area and downloadable as CSV, alongside the generated reports. If a figure on any screen needs to travel into a spreadsheet or a slide, this is where it leaves the dashboard.

Power BI

Who sees it: analyst, evaluator and admin.

The Power BI report built over the same data, embedded in the dashboard. It needs an internet connection to load.

Who can see what

The same matrix is enforced in three places — the sidebar, the route guards, and the FastAPI endpoints — so the interface is never the only gate.

Screen	Operator	Analyst	Evaluator	Admin
Executive Dashboard	✓	✓	✓	✓
Live Network Map	✓	✓	✓	✓
Passenger Flow	✓	✓	✓	✓
Route Performance	✓	✓	✓	✓

Screen	Operator	Analyst	Evaluator	Admin
Delay Intelligence	✓	✓	✓	✓
Occupancy & Crowding	✓	✓	✓	✓
Demand Forecast	✓	✓	✓	✓
Anomaly Detection	✓	✓	✓	✓
Recommendations	✓	✓	✓	✓
What-If Simulator	✓	✓	✓	✓
Spark vs Python	—	✓	✓	✓
Data Quality	—	✓	✓	✓
Reports & Export	—	✓	✓	✓
Power BI	—	✓	✓	✓
HDFS & Spark Jobs	—	—	✓	✓
User Management	—	—	—	✓

On a phone

Every screen works at phone width. The sidebar collapses behind the menu button at the top left; tap it to open the navigation and tap again to close it. Wide tables scroll sideways inside their own panel rather than stretching the page, so the layout never scrolls horizontally as a whole.

If something looks wrong

- **“Backend offline”** — the sign-in page says so and offers the demo accounts. Start the backend (uvicorn, port 8000) for real sign-in, account creation and exports.
- **The map or weather chip is blank** — the map tiles and the weather chip come from public services and need an internet connection, even when the rest runs locally.
- **Data does not load locally** — the frontend reads the API address from VITE_API_BASE and falls back to http://localhost:8000. If the backend runs elsewhere, set that variable and restart the dev server.
- **A screen is missing from the sidebar** — a role that cannot open a screen does not see its sidebar entry, and typing its URL redirects to the Executive Dashboard. Sign in as evaluator (or admin) to reach everything.